

Assignment 13

Shahriar Hasan

August 4, 2025

Objective

- Generating synthetic faces using Deep Convolutional GAN.
- Generating synthetic faces with long hair using Conditional GAN.
- Generating a painted form of faces using CycleGAN.
- Figuring out the reason behind using the BCE loss instead of the MiniMax loss in Tensorflow implementation (mathematical derivation).

1. Generating Synthetic Faces Using Deep Convolutional GAN

1.1. Dataset

- **Dataset Name:** CelebA (CelebFaces Attributes Dataset)
- **Source:** Kaggle: jessicali9530/celeba-dataset
- **Images Used:** 20,000 samples from `img_align_celeba/img_align_celeba` folder
- **Image Size:** Resized to 64×64 pixels
- **Normalization:** Pixel values scaled to the range $[-1, 1]$ using:

$$\text{normalized_img} = \frac{\text{img} - 127.5}{127.5}$$

1.2. DCGAN Architecture

The DCGAN consists of two adversarial networks: the Generator and the Discriminator.

1.2.1 Generator

- Input: 100-dimensional random noise vector
- Layers:
 - Dense layer followed by BatchNorm and ReLU
 - Three Conv2DTranspose layers for upsampling
 - Output activation: `tanh`
- Output: 64×64×3 RGB synthetic image

1.2.2 Discriminator

- Input: $64 \times 64 \times 3$ image
- Layers:
 - Two Conv2D layers with LeakyReLU and Dropout
 - Flatten layer followed by Dense
- Output: Single logit score (real or fake)

1.3. Training Details

Parameter	Value
Epochs	50
Batch Size	128
Noise Vector Dimension	100
Optimizer	Adam ($\beta_1 = 0.5$)
Learning Rate	$1e-4$
Loss Function	Binary Cross-Entropy (from_logits=True)

Table 1: DCGAN Training Parameters

1.4. Training Workflow

1. Preprocess images by resizing and normalizing them.
2. Construct dataset using `tf.data.Dataset`, shuffle and batch it.
3. In each training step:
 - Generate fake images from random noise
 - Compute discriminator loss on real and fake images
 - Compute generator loss
 - Backpropagate and update both networks
4. After every epoch, generate and display synthetic face images.

1.5. Results and Visualization

1.5.1 Real Training Samples

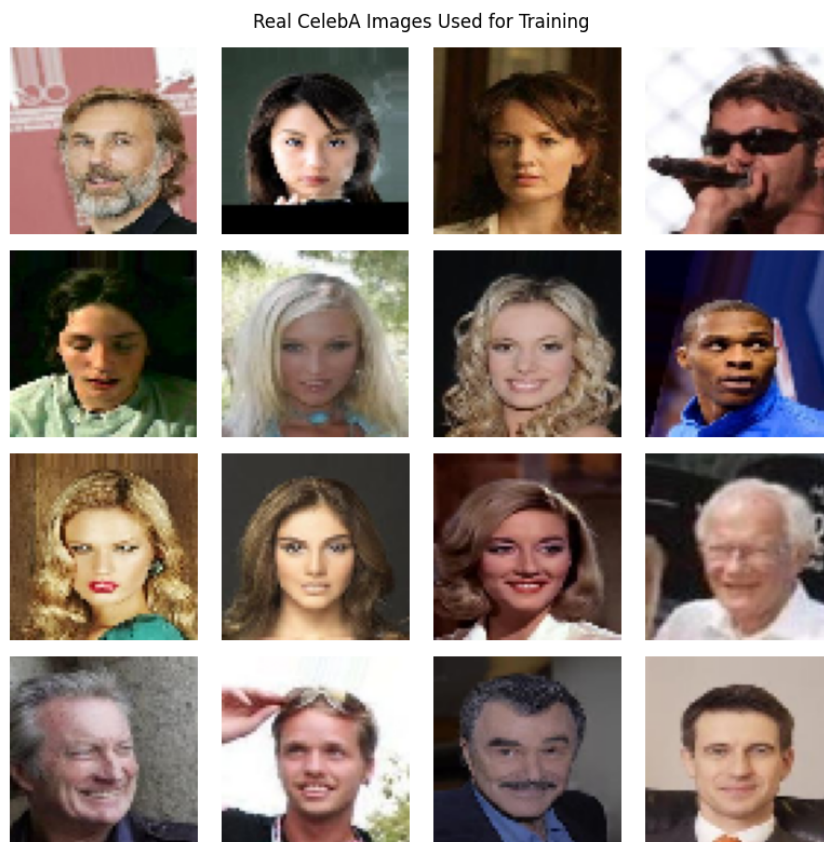


Figure 1: Real CelebA Images Used for Training

1.5.2 Generated Faces After Training

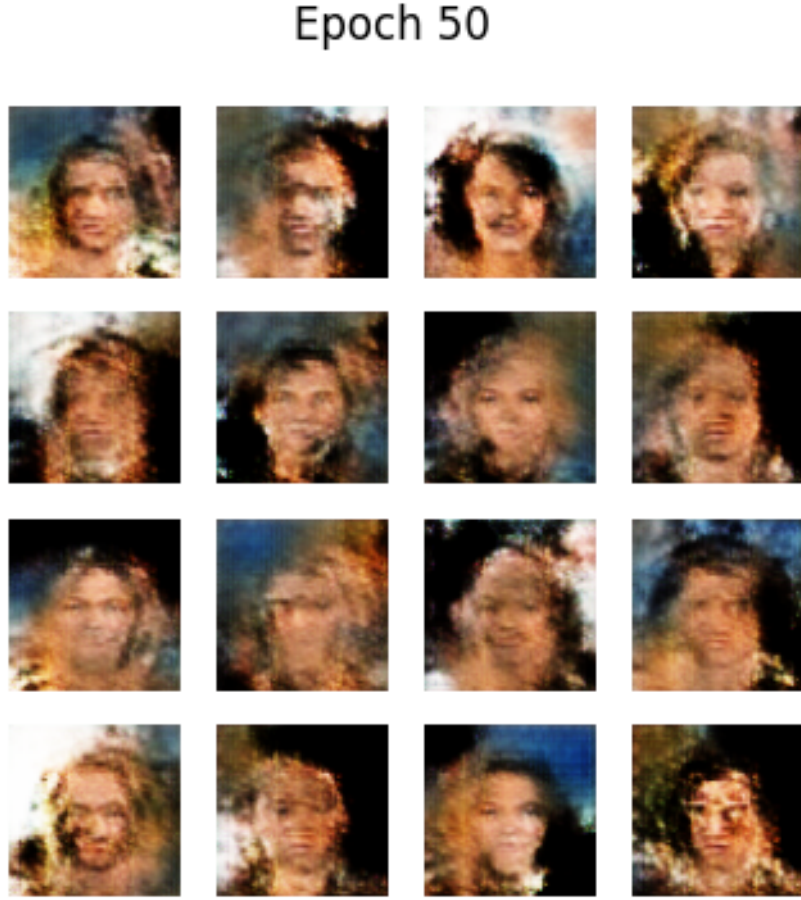


Figure 2: Synthetic Faces Generated After 50 Epochs

1.6. Evaluation

The generated images improve in quality over epochs, transitioning from noise to realistic human faces.

2. Generating synthetic faces with long hair using Conditional GAN

This objective focuses on generating synthetic facial images with long hair using a Conditional Generative Adversarial Network (CGAN). We use the CelebA dataset and apply conditioning based on hair attributes (wavy or straight). The generator and discriminator are conditioned on the hair attribute to produce and verify realistic long-haired face images.

2.1. Dataset

We use the CelebA dataset, which contains over 200,000 annotated celebrity face images. We extract attributes from `list_attr_celeba.csv` and define a binary attribute ‘Long_Hair’ based on the presence of ‘Wavy_Hair’ or ‘Straight_Hair’.

2.1.1 Data Samples

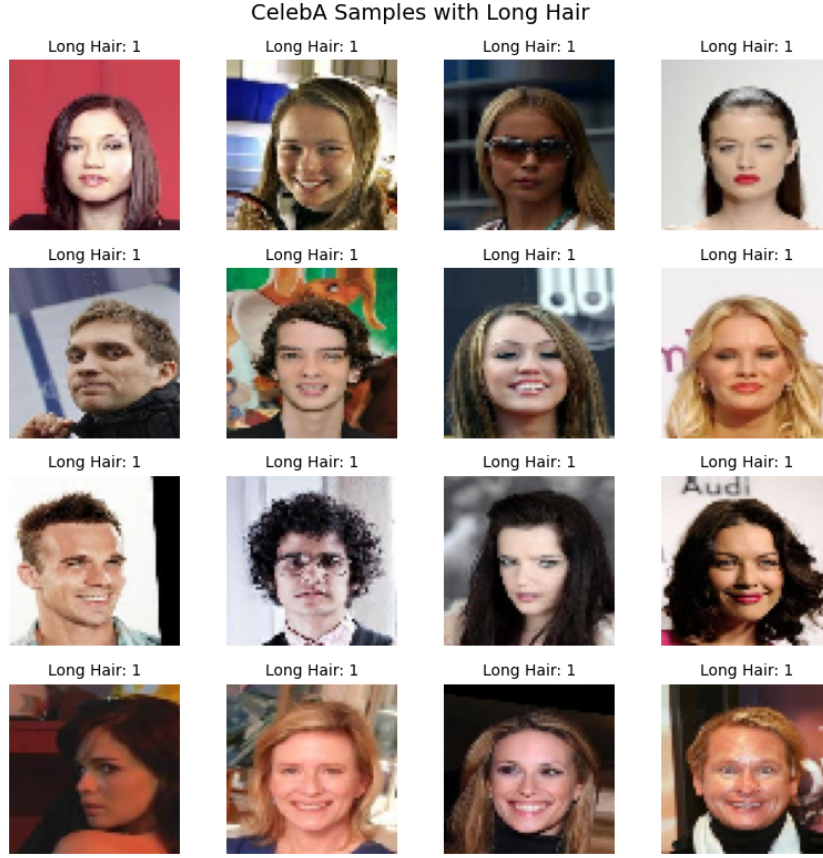


Figure 3: CelebA Samples with Long Hair

2.1.2 Preprocessing

- Images are resized to 64×64 pixels.
- Pixel values are normalized to the range $[-1, 1]$.
- 10,000 images with long hair are selected.

2.2. Methodology

2.2.1 Conditional Generator

The generator takes as input a 100-dimensional noise vector and a binary hair label. It embeds the label and concatenates it with the noise, then applies several transposed convolutional layers to generate a $64 \times 64 \times 3$ RGB image.

2.2.2 Conditional Discriminator

The discriminator takes as input an image and a hair label. It embeds the label and concatenates it with the image along the channel axis. The model then uses convolutional layers to output a single real/fake score.

2.2.3 Loss Functions

- Generator loss: encourages the generator to produce realistic images that fool the discriminator.
- Discriminator loss: encourages the discriminator to distinguish real images from generated ones.

2.2.4 Training

- Optimizer: Adam ($\text{lr} = 1e-4$)
- Batch size: 128
- Epochs: 100
- Images generated every epoch for visualization

2.3. Results

The CGAN model was trained for 50 epochs. After each epoch, 16 images were generated using noise vectors and the long-hair label.

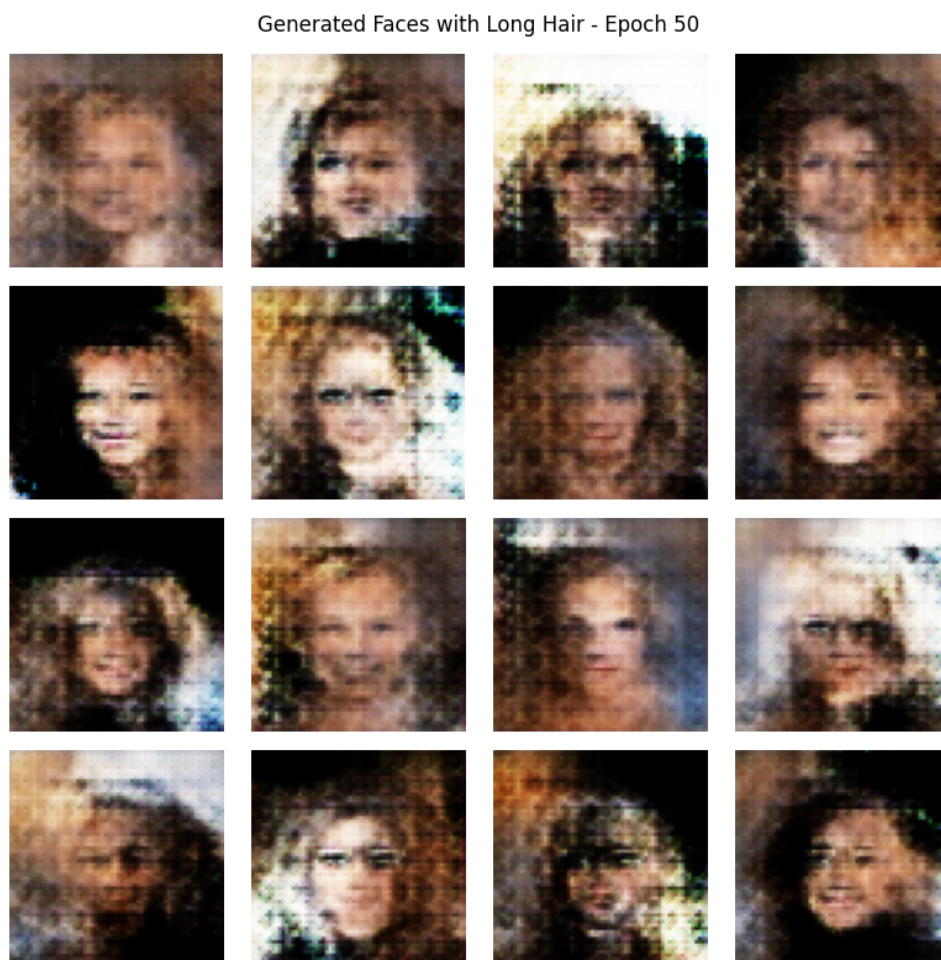


Figure 4: Sample Generated Faces with Long Hair (Epoch 50)

The quality of generated images improved over time, showing clearer facial features and realistic hair textures.

3. Generating painted form of faces using CycleGAN

3.1. Datasets

Dataset	Source	Samples Used	Description
CelebA	Kaggle	1000	Real human face images
Van Gogh Paintings	Kaggle	1000	Paintings by Vincent Van Gogh

Table 2: Datasets Used

3.1.1 Data Samples



Figure 5: CelebA Samples and Van Gogh Paintings Samples

3.1.2 Preprocessing:

- All images resized to 128×128 pixels.
- Normalized to the range $[-1, 1]$.
- Handled with OpenCV.

3.2. Methodology

3.2.1 Model Overview

CycleGAN consists of:

- Two Generators: $G : X \rightarrow Y$ (CelebA $\rightarrow$ Van Gogh) and $F : Y \rightarrow X$ (Van Gogh $\rightarrow$ CelebA)
- Two Discriminators: D_X (for CelebA) and D_Y (for Van Gogh)

3.2.2 Loss Functions

- **Adversarial Loss:** Least Squares GAN (MSE)
- **Cycle-consistency Loss:** Mean Absolute Error
- **Identity Loss:** Mean Absolute Error to preserve structure

Final Generator Loss:

$$\mathcal{L}_G = \mathcal{L}_{GAN} + 10 \cdot \mathcal{L}_{cycle} + 5 \cdot \mathcal{L}_{identity}$$

3.2.3 Architecture

Generator:

- Downsampling: Conv2D (stride 2)
- 3 ResNet Blocks
- Upsampling: Conv2DTranspose
- Output: Tanh activation

Discriminator:

- PatchGAN: Conv2D layers (stride 2), LeakyReLU
- Output: Single channel feature map

3.3. Training

Hyperparameters:

- Image Size: 128×128
- Batch Size: 16
- Epochs: 10
- Optimizer: Adam ($\beta_1 = 0.5$)
- Learning Rate: 2×10^{-4}

Training Strategy:

- Each batch from CelebA and Van Gogh is processed.
- Generator output is cycled through the reverse generator.
- Losses are computed and backpropagated with `tf.GradientTape`.

3.4. Results

Example Results (CelebA to Van Gogh):

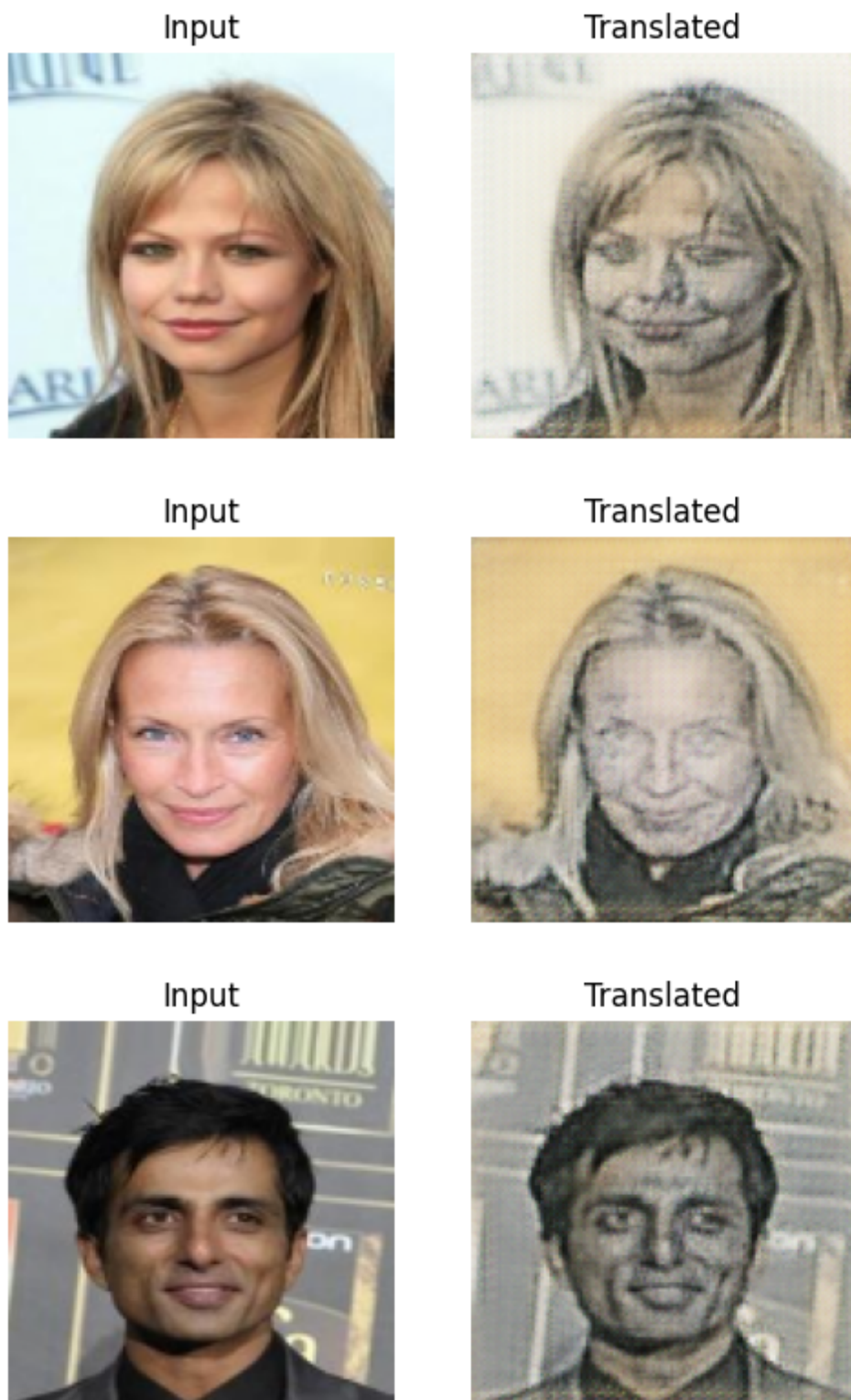


Figure 6: Sample Translation Result after epoch 10

3.5. Observations:

- The model begins to apply Van Gogh-style color patterns and textures.
- Facial structure is moderately preserved.
- More epochs and deeper networks would improve fidelity.

4. Figure out the reason behind using the BCE loss instead of the MiniMax loss in Tensorflow implementation

4.1. Original MiniMax Loss (Goodfellow et al., 2014)

Generative Adversarial Networks (GANs) are based on a minimax game between a generator G and a discriminator D . The original loss function is defined as:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (1)$$

Here:

- x is a real data sample from distribution $p_{\text{data}}(x)$,
- z is a latent variable sampled from a prior noise distribution $p_z(z)$,
- $G(z)$ is a generated sample,
- $D(x)$ outputs the probability that x is real.

4.2. Discriminator Loss (MiniMax)

Given a fixed generator G , the discriminator's objective is to maximize the following:

$$L_D = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (2)$$

This is equivalent to the binary cross-entropy loss where:

- Real samples are labeled with 1,
- Fake (generated) samples are labeled with 0.

4.3. Generator Loss (MiniMax)

The generator tries to fool the discriminator, so its objective is to minimize:

$$L_G^{\text{minimax}} = \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (3)$$

Problem: When the discriminator is strong early in training, $D(G(z)) \approx 0$, and hence $\log(1 - D(G(z))) \approx 0$, leading to vanishing gradients and slow generator learning.

4.4. Heuristic Generator Loss (Non-Saturating)

To overcome vanishing gradients, the following alternative loss was proposed:

$$L_G^{\text{heuristic}} = -\mathbb{E}_{z \sim p_z}[\log D(G(z))] \quad (4)$$

This loss provides stronger gradients when $D(G(z)) \rightarrow 0$ and leads to more stable training.

4.5. BCE Loss in TensorFlow

TensorFlow implements GAN losses using the Binary Cross-Entropy (BCE) loss:

$$\text{BCE}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad (5)$$

Discriminator Loss:

$$L_D = \mathbb{E}_{x \sim p_{\text{data}}}[\text{BCE}(1, D(x))] + \mathbb{E}_{z \sim p_z}[\text{BCE}(0, D(G(z)))] \quad (6)$$

Generator Loss:

$$L_G = \mathbb{E}_{z \sim p_z}[\text{BCE}(1, D(G(z)))] \quad (7)$$

This corresponds to the non-saturating heuristic loss.

4.6. Gradient Comparison

4.7. MiniMax Gradient

$$\nabla_{\theta} \mathbb{E}_z[\log(1 - D(G(z)))] \rightarrow 0 \quad \text{as } D(G(z)) \rightarrow 0 \quad (8)$$

4.8. Heuristic (BCE) Gradient

$$\nabla_{\theta} \mathbb{E}_z[-\log D(G(z))] \text{ is large when } D(G(z)) \rightarrow 0 \quad (9)$$

Hence, BCE yields better learning dynamics, especially early in training.

4.9. Summary

Aspect	MiniMax Loss	BCE (Heuristic) Loss
Generator loss	$\log(1 - D(G(z)))$	$-\log D(G(z))$
Training stability	Poor early on	More stable
Gradient magnitude	Vanishes when $D(G(z)) \rightarrow 0$	Strong gradients
Used in TensorFlow		

4.10. TensorFlow Example

```
loss_fn = tf.keras.losses.BinaryCrossentropy(from_logits=False)

# Discriminator loss
real_loss = loss_fn(tf.ones_like(real_output), real_output)
fake_loss = loss_fn(tf.zeros_like(fake_output), fake_output)
d_loss = real_loss + fake_loss

# Generator loss
g_loss = loss_fn(tf.ones_like(fake_output), fake_output)
```

Source:

You can find the full code implementation on my GitHub or you can directly run it on Google Colab.